DA6401W - Assignment 5

Department of Data Science and AI, IIT Madras April 2026

Submitted by: Sohang Chopra <DA25M622>

Problem 1: Numerical: Normalization Dynamics and Gradient Flow in Deep CNN

Consider a convolutional neural network receiving a multi-channel input of size $64 \times 64 \times 3$. The network performs the following sequence of operations:

1. Convolution Layer 1 Filters = 16, Kernel = 7×7, Stride = 2, Padding = 3
2. Batch Normalization
3. Max Pooling Pool size = 3×3, Stride = 2, Padding = 1
4. Residual Block consisting of two convolution layers Kernel = 3×3, stride = 1, padding = 1, filters = 16 Identity skip connection.
5. Global Average Pooling followed by Fully Connected layer.

Answer the following:

1. Compute feature map dimension after each stage.
2. BatchNorm statistics: $\mu = 12, \sigma^2 = 16, \gamma = 0.5, \beta = -1$. Find normalized output for activation $x = 20$.
3. Residual mapping produces $F(x) = \begin{bmatrix} 2 \\ -1 \\ 0.5 \end{bmatrix}, x = \begin{bmatrix} 1 \\ 3 \\ -0.5 \end{bmatrix}$. Compute block output.
4. Gradient clipping with threshold $c = 5$. If gradient norm is $\|g\|_2 = 25$, find:
   - clipped gradient vector expression
   - new gradient norm
5. Min-max normalization range $[-4, 28]$. Compute normalized value for $x = 20$.

Solution 1

Feature Map Dimension after:

- Convolution layer: $\lfloor \frac{W-F+2P}{S} \rfloor + 1, \lfloor \frac{H-F+2P}{S} \rfloor + 1, K$ where W, H are input width, height, F is filter kernel size, P, S are padding, stride, K is no. of filters in layer.
- Pooling layer: similar to convolution but depth doesn't change: $\lfloor \frac{W-F+2P}{S} \rfloor + 1, \lfloor \frac{H-F+2P}{S} \rfloor + 1, D$ where D is input depth
- Global Average Pooling: $(D,)$ (i.e. only 1 dimensional vector of size depth remains)

Batch Norm output formula: $y = \beta + \gamma(x - \mu)/(\sigma + \epsilon)$ where ϵ is for numerical stability.

Residual block output: $F(x) + x$

Gradient Clipping vector formula (clips only if L2 norm > max threshold, else leaves unchanged): $c\frac{g}{\|g\|_2}$ where c is threshold, g is gradient.

Min-Max normalization formula (output is in [0,1]): $(x - minval)/(maxval - minval)$

1. Feature map dimensions after each layer are:

- Convolution Layer 1: $(floor((64 - 7 + 2 * 3)/2) + 1, floor((64 - 7 + 2 * 3)/2) + 1, 16) = (32, 32, 16)$
- Batch Normalization (unchanged data shape): $(32, 32, 16)$
- Max Pooling: $(floor((32 - 3 + 2 * 1)/2) + 1, floor((32 - 3 + 2 * 1)/2) + 1, 16) = (16, 16, 16)$
- Residual Block: input is added to final output due to skip connection, so feature map size is unchanged: $(16, 16, 16)$
- Global Average Pooling followed by Fully Connected Layer: $(16,)$

2. BatchNorm output: $\sigma = sqrt(16) = 4$, so $-1 + 0.5 * (20 - 12)/4 = 0$
3. Residual Block output: $(2, -1, 0.5)^T + (1, 3, -0.5)^T = (3, 2, 0)^T$
4. Clipped gradient vector expression: $5g/25 = g/5$ where g is gradient vector (since gradient norm 25 is more than threshold 5). New gradient norm is 5.
5. Min-max normalized value: $(20 + 4)/(28 + 4) = 0.75$

Problem 3

Training deep neural networks often involves stabilizing both forward activations and backward gradients. Two commonly used techniques are **normalization methods** (Batch Normalization, Layer Normalization) and **gradient clipping** .

1. **Batch Normalization vs Layer Normalization**: Consider an input tensor $X \in \mathrm{R}^{B \times d}$, where B is the batch size and d is the feature dimension.
    - Write the mathematical formulation of **Batch Normalization** and clearly indicate along which dimension mean and variance are computed.
    - Write the formulation of **Layer Normalization** and specify the normalization dimensions.
    - Explain why Batch Normalization performance degrades when the batch size is very small, whereas Layer Normalization does not suffer from this issue.
    - In the context of sequence models (e.g., Transformers), explain why Layer Normalization is preferred over Batch Normalization.
2. **Effect on Optimization Dynamics**
    - Explain how normalization methods affect the conditioning of the optimization problem.
    - Discuss the role of normalization in mitigating *internal covariate shift* . Is this explanation sufficient to justify their effectiveness?
3. **Gradient Clipping**: During training, suppose the gradient vector $g \in \mathrm{R}^n$ has a very large norm.
    - Define **gradient clipping by norm** and write its mathematical formulation.

- Show that after clipping with threshold c, the resulting gradient has norm at most c.
- Explain how gradient clipping helps in training recurrent or very deep networks.

Solution 3

1. **Batch Normalization vs Layer Normalization**:
   - Batch Normalization formula (along batch dimension B for the input feature): $y_i = \beta + \gamma(x_i - \mu)/(\sigma + \epsilon)$ where μ, σ are mean, standard deviation (learned parameters of training data feature x), ϵ is for numerical stability to avoid division by 0.
   - Layer Normalization formula is same as BatchNorm, but done along feature dimension d not batch: $y_i = \beta + \gamma(x_i - \mu)/(\sigma + \epsilon)$
   - In Batch Norm, for each feature values are normalized according to mean and standard deviation across whole batch. When batch size of each mini-batch is small, mean and standard deviation are very unstable and poorly approximate overall mean, standard deviation, so poor normalization happens. Layer Norm is not affected as it's averaged along feature dimension independent of batch size.
   - Sequence models like RNN, Transformers work with sequences that can be of varying length. Batch Norm normalizes over each feature (position in sequence) for all sequences, but this makes it unstable as values are missing in some samples that have shorter sequence length. Layer Norm normalizes over each sequence so it doesn't matter how long any other sequence is. This is why LayerNorm is preferred.
2. **Effect on Optimization Dynamics**
   - *Condition Number* is ratio $\lambda_{max}/\lambda_{min}$ (largest and smallest eigenvalues of Hessian matrix of loss). Due to normalization, features have similar scales, loss surface becomes more spherical, condition number reduces. It allows higher learning rates and faster convergence without the model oscillating in narrow loss valleys.
   - *Internal Covariance Shift* refers to the fact that layers' input distributions are constantly changing due to previous layers' weights getting updated. Batch Norm prevents this by forcing values to have a fixed mean 0, variance 1. But this is insufficient to explain Batch Norm's effectiveness, as latest research suggests they work well even when ICS is injected, and so their effectiveness likely has more to do with loss surface smoothening effect.
3. **Gradient Clipping**:
   - If L2 norm is within threshold then gradient remains unchanged, otherwise Gradient Clipping by Norm de-scales gradient L2 norm to maximum threshold norm while maintaining same direction (unit vector). Clipped gradient vector is $c\frac{g}{\|g\|_2}$ as gradient norm is larger than threshold.
   - If L2 norm is within threshold, then $\|g\|_2 < c$, else $\left\|c\frac{g}{\|g\|_2}\right\| = c$. So clipped gradient norm is at most c.
   - In backpropogation, weight matrices of each layer get multiplied while calculating gradients. In very deep networks, if weights are > 1, then they explode

(become very large) by the time gradients reach starting layers, since $x^{\infty} = \infty$ for any $x > 1$. Similarly in RNN (Recurrent Neural Network), the hidden state depends on previous hidden state $h_t = \sigma(Wh_{t-1} + Ux_1)$ - so during backprop, same weights matrix W is repeatedly multiplied by itself to calculate gradient causing gradient to blow up if weights > 1. Since gradient clipping caps maximum gradient norm, it prevents this "Exploding Gradient" problem.

Problem 5: Numerical: Effect of Stride and Padding in Convolution

Consider an input image of size $64 \times 64 \times 3$. A convolution layer is applied with the following parameters:

- Number of filters = 32
- Kernel size = 5×5
- Stride = 2
- Padding = 2

1. Compute the spatial dimensions of the output feature map.
2. Compute the total number of learnable parameters in the convolution layer (including bias).

Solution 5

1. Output feature map dimensions are: $(floor((W - F + 2P)/S) + 1, floor((H - F + 2P)/S) + 1, K) = (floor((64 - 5 + 2 * 2)/2) + 1, floor((64 - 5 + 2 * 2)/2) + 1, 16) = (32, 32, 16)$
2. Each kernel has a $(5, 5, 3)$ weight matrix. And there's one bias value per filter. So no. of learnable parameters is $32 * (5 * 5 * 3 + 1) = 2,432$

Problem 6: Conceptual: Residual Connections and Optimization

Consider two neural networks with identical depth:

- Network A: Plain CNN
- Network B: Residual CNN with skip connections

A residual block computes

$$y = F(x) + x$$

Answer the following:

1. Explain why residual connections improve gradient flow during backpropagation.
2. Suppose $F(x) = 0$. What is the output of the residual block? What does this imply about training very deep networks?

Solution 6

1. Residual connections prevent "Vanishing Gradients" problem by providing gradients an easier alternative path to flow backward (via skip connection) instead of complex non-linear transformations on other branch of residual block.
2. Output of residual block is $y = x$. This implies that residual block allowed model to learn identity mapping (normally layer struggles with this) - it allowed model to "skip" the residual block when it wasn't required for some input data. So model may "skip" residual block for an easy input but not for a harder input.

Problem 9 Numerical: Large Kernel vs Small Kernels with Pooling

Consider an input feature map $X \in R^{64\times64\times3}$.

Two different designs are applied:

Design A (Single Large Convolution):

- Kernel size = 7×7
- Number of filters = 64
- Stride = 2, Padding = 3

Design B (Smaller Convolutions + Pooling):

- First layer: 3×3, 32 filters, stride = 1, padding = 1
- Max Pooling: 2×2, stride = 2
- Second layer: 3×3, 64 filters, stride = 1, padding = 1

Tasks:

1. Compute the output feature map dimensions for:
    - Design A
    - Design B (after all layers)
2. Compute the total number of learnable parameters (including bias) for Design A.
3. Compute the total number of learnable parameters (including bias) for Design B.
4. Compute the ratio:

$$\frac{\text{Parameters in Design B}}{\text{Parameters in Design A}}$$

Solution 9

1. Output feature map dimensions for models:
    - Model A: $(floor((64 - 7 + 2 * 3)/2) + 1, floor((64 - 7 + 2 * 3)/2) + 1, 64) = (32, 32, 64)$
    - Model B:
        - Conv 1: $(floor((64 - 3 + 2 * 1)/1) + 1, floor((64 - 3 + 2 * 1)/1) + 1, 32) = (64, 64, 32)$

    - Max Pool: $(floor((64 - 2 + 2 * 0)/2 + 1), floor((64 - 2 + 2 * 0)/2 + 1), 32) = (32, 32, 32)$
    - Conv 2 (final output): $(floor((32 - 3 + 2 * 1)/1) + 1, floor((32 - 3 + 2 * 1)/1) + 1, 64) = (32, 32, 64)$
2. No. of learnable parameters in A = $K(F^2D + 1) = 64 * (7^2 * 3 + 1) = 9,472$
3. No. of learnable parameters in B (maxpool has no learnable parameters so sum of conv learnable parameters): $32 * (3^2 * 3 + 1) + 64 * (3^2 * 32 + 1) = 19,392$
4. Ratio (params in B / params in A) = $19392/9472 \approx 2.05$

Problem 11 (Theoretical Question - Weight Initialization in CNNs with ReLU)

Consider the following CNN architecture trained for image classification with inputs shape (batch_size, a, b, 3):

Fan-in and Fan-out Definitions:

- **Convolutional layers:** both fan_in and fan_out use the full receptive field area $k \times k$, multiplied by the respective channel count:

$$fanin = C_{in} \times k \times k, \quad fanout = C_{out} \times k \times k$$

- **Fully connected layers:**

fanin = number of input neurons , fanout = number of output neurons

Layer	Type	Details	Activation
1	Conv2D	kernel = 3×3, in_ch = 3, out_ch = 64	ReLU
2	BatchNorm + MaxPool	2×2 pool	_
3	Conv2D	kernel = 3×3, in_ch = 64, out_ch = 128	ReLU
4	BatchNorm + MaxPool	2×2 pool	_
5	Conv2D	kernel = 3×3, in_ch = 128, out_ch = 256	ReLU
6	Flatten	$4 \times 4 \times 256 = 4096$	_
7	FC (Linear)	4096 --> 1024	ReLU
8	FC (Linear)	1024 --> 10	Softmax

- State the **Kaiming He Normal** initialization formula. Define every term clearly, including how fan in is computed for a convolutional layer.
- State the **Kaiming He Uniform** initialization formula and explain how it relates to the normal variant.
- For each **convolutional layer** in the network above, compute fanin, fanout, the standard deviation σ (normal), and the bound a (uniform).
- For each **fully connected layer** in the network above, compute fanin, fanout, σ, and a .

Solution 11

- Kaimeng He Normal initialization: $W \sim N(\mu = 0, \sigma = \sqrt{2/fanin})$ where fanin for a convolutional layer is calculated using no. of channels in input data (multiplied to remaining input dimensions): $fanin = C_{in} \times k \times k$
- Kaimeng He Uniform initialization: $W \sim U(\pm\sqrt{6/fanin})$. It's related to Normal variant as both probability distributions have same expected weight value (0) and same distribution variance.
- Calculating for Conv2D and FC (Linear) layers:

Layer	Type	Details	Fanin	Fanout	Normal σ	Uniform a
1	Conv2D	$3 * 3 * 3 = 27$	27	576	$\sqrt{2/27} = 0.2722$	$\sqrt{6/27} = 0.4714$
2	BatchNorm + MaxPool	$2 * 2$ pool	-	-	-	-
3	Conv2D	$64 * 3 * 3 = 576$	576	1152	$\sqrt{2/576} = 0.0589$	$\sqrt{6/576} = 0.1021$
4	BatchNorm + MaxPool	$2 * 2$ pool	-	-	-	-
5	Conv2D	$128 * 3 * 3 = 1152$	1152	2304	$\sqrt{2/1152} = 0.0417$	$\sqrt{6/1152} = 0.0722$
6	Flatten	$4 * 4 * 256 = 4096$	-	-	-	-
7	FC (Linear)	$4096 \rightarrow 1024$	4096	1024	$\sqrt{2/4096} = 0.0221$	$\sqrt{6/4096} = 0.0383$
8	FC (Linear)	$1024 \rightarrow 10$	1024	10	$\sqrt{2/1024} = 0.0442$	$\sqrt{6/1024} = 0.0765$

Problem 12: Hyperparameter Optimization Strategies

1. How does Bayesian Optimization differ from grid search and random search in its approach to hyperparameter tuning? What key advantage does it offer?
2. Why cannot Bayesian Optimization replace gradient descent for training neural networks?
3. Why cannot gradient descent be used directly for hyperparameter tuning? What property of the hyperparameter optimization problem makes it fundamentally different from model training?

Solution 12

1. Unlike Grid Search and Random Search (uninformed strategies that treat all hyper-parameter combinations as independent), Bayesian Optimization assumes hyper-parameters follow a distribution (say Gaussian). It learns parameters of the assumed

distribution (like mean, variance) gradually with each sample, and samples subsequent hyper-parameters from this distribution (to train model & validate performance for finding best). It has the advantage that, assuming the hyper-parameters do follow this prior distribution, most of the time sampled hyper-parameters are most likely (near peak), and very little time is wasted on unlikely hyper-parameter combinations (in extreme tails of Gaussian distribution). This saves computation resources.

2. Bayesian Optimization is a black-box technique (it uses only outputs not internal model structure), and at the scale of millions / billions of parameters in neural networks, it would get very very slow. On the other hand, gradient descent is better for neural network training as it precisely updates weights using gradients to point approximately towards optimal point.
3. Gradient Descent requires a differentiable objective function, but for hyper-parameter tuning the validation loss is generally not differentiable or the gradient would be very difficult to calculate (as we would need to backpropogate gradients with respect to hyper-parameters through the entire model training process). That's why Bayesian Optimization is used instead.

Problem 13

In an image segmentation task, the goal is to separate pets from the background. The ground truth labels are provided as a tensor:

$$Y \in \mathrm{R}^{H \times W \times 1}$$

where each pixel takes a value in $\{0, 1\}$, with 1 indicating the presence of a pet and 0 indicating background.

Now consider extending this task to identify additional elements in the image such as trees, roads, and other objects.

1. **Label Representation**
    - Do you think the same labeling strategy (single-channel binary mask) would be sufficient for this extended task? Justify your answer.
    - If not, propose a suitable labeling strategy. Clearly describe:
        - how each pixel should be represented,
        - the shape of the label tensor.
    - What should be the shape of the model output for this task?
2. **Loss Function**
    - What loss function would you use for this problem? Write its mathematical form.
    - In practice, datasets may have imbalance between different regions (e.g., large background vs small objects). Suggest an additional loss (or modification) to address this and briefly explain why.

Solution 13

1. **Label Representation**
    - No single-channel binary mask would not be sufficient for extended tasks, as we now instead of 1 type of object (present or absent), we need to identify many classes of objects like trees, roads, etc. which cannot be encoded with just 0 or 1 per pixel.
    - For each pixel, we could have a one-hot encoding of each class (1/0 for each class index to indicate presence or absence). Label tensor (per pixel) would be of size $(C,)$ where C is no. of classes of objects that we want to identify.
    - Shape of model output tensor would be (W, H, D, C) where W, H, D are width, height, depth (aka channels) of input image.

2. **Loss function**
    - Multi-class Cross Entropy loss can be used.
    - *Weighted Cross Entropy* can be used - a loss modification added to handle imbalance that rewards true positives (i.e. correctly identify presence of desired object class) much more than true negatives (i.e. correctly identify background / absence of object). So modified binary cross entropy loss becomes (for a single object class) (terms divided by no. of elemennts in respective class: presence or absence of desired object):

$$\sum -\frac{y_i \ln(\hat{y_i})}{N_{object}} - \frac{(1-y_i)\ln(1-\hat{y_i})}{N_{background}}$$